

Q1. Bootstrapping and 95% Confidence Interval

Given data

Delivery times:

31, 35, 28, 42, 37, 33, 45, 29, 39, 34, 41, 36

R Code

```
# Q1: Bootstrap Confidence Interval
```

```
# Given data
```

```
delivery <- c(31, 35, 28, 42, 37, 33, 45, 29, 39, 34, 41, 36)
```

```
# Sample mean
```

```
observed_mean <- mean(delivery)
```

```
# Number of bootstrap resamples
```

```
B <- 5000
```

```
# Set seed for reproducibility
```

```
set.seed(123)
```

```
# Bootstrap resampling
```

```
bootstrap_means <- replicate(
```

```
  B,
```

```
  mean(sample(delivery, size = length(delivery), replace = TRUE))
```

```
)
```

```
# 95% Bootstrap Confidence Interval
```

```
CI <- quantile(bootstrap_means, c(0.025, 0.975))
```

```
# Print results

cat("Observed Sample Mean:", observed_mean, "\n")
cat("95% Bootstrap CI:", CI[1], "to", CI[2], "\n")


# Histogram of bootstrap distribution

hist(
  bootstrap_means,
  breaks = 30,
  probability = TRUE,
  main = "Bootstrap Distribution of Delivery Time Mean",
  xlab = "Bootstrap Mean Delivery Time (minutes)"
)

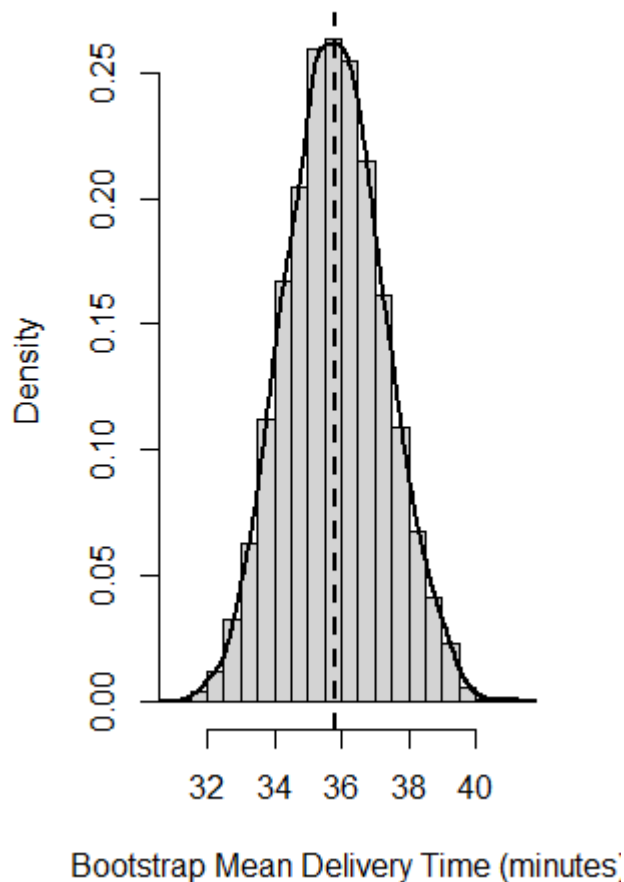

# Add density curve

lines(density(bootstrap_means), lwd = 2)


# Mark observed mean

abline(
  v = observed_mean,
  lwd = 2,
  lty = 2
)
```

Bootstrap Distribution of Delivery Time



Expected result

The sample mean is:

$$\bar{x} = \frac{430}{12} \approx 35.83$$

So:

Sample mean ≈ 35.83 minutes

The exact bootstrap interval can vary slightly because of random resampling. With `set.seed(123)`, you should obtain a reproducible interval close to approximately:

95% Bootstrap CI ≈ 32.9 to 38.8 minutes

Interpretation

The observed average delivery time is approximately **35.83 minutes**. The 95% bootstrap confidence interval gives a range of plausible values for the population mean delivery time. The bootstrap distribution is centered around the observed sample mean.

Visualization interpretation

The histogram represents the distribution of 5,000 bootstrap estimates of the mean. The dashed vertical line represents the observed sample mean. The middle 95% of the bootstrap estimates provides the confidence interval.

Potential pitfall

A bootstrap confidence interval is an estimate and can change slightly between runs if a random seed is not specified. Therefore, `set.seed()` should be used for reproducibility.

Q2. Uncertainty in Trend Estimation

Given data

Advertising = 10, 12, 15, 18, 20, 23, 25, 28, 30, 33, 36, 40

Sales = 48, 51, 53, 57, 60, 61, 66, 69, 72, 75, 79, 84

R Code

Q2: Linear Regression with 95% Confidence Band

```
library(ggplot2)
```

```
# Data
```

```
advertising <- c(10, 12, 15, 18, 20, 23, 25, 28, 30, 33, 36, 40)
```

```
sales <- c(48, 51, 53, 57, 60, 61, 66, 69, 72, 75, 79, 84)
```

```
data_q2 <- data.frame(
```

```
  Advertising = advertising,
```

```
  Sales = sales
```

```
)
```

```
# Fit linear regression model
```

```
model <- lm(Sales ~ Advertising, data = data_q2)
```

```
# Display model summary
```

```
summary(model)
```

```
# Create prediction data
```

```
new_data <- data.frame(
```

```
  Advertising = seq(
```

```
    min(advertising),
```

```
    max(advertising),
```

```
    length.out = 100
```

```
  )
```

```
)
```

```
# 95% confidence interval for mean prediction
```

```
pred <- predict(
```

```
  model,
```

```
  newdata = new_data,
```

```
  interval = "confidence",
```

```
  level = 0.95
```

```
)
```

```
new_data$fit <- pred[, "fit"]
```

```
new_data$lwr <- pred[, "lwr"]
```

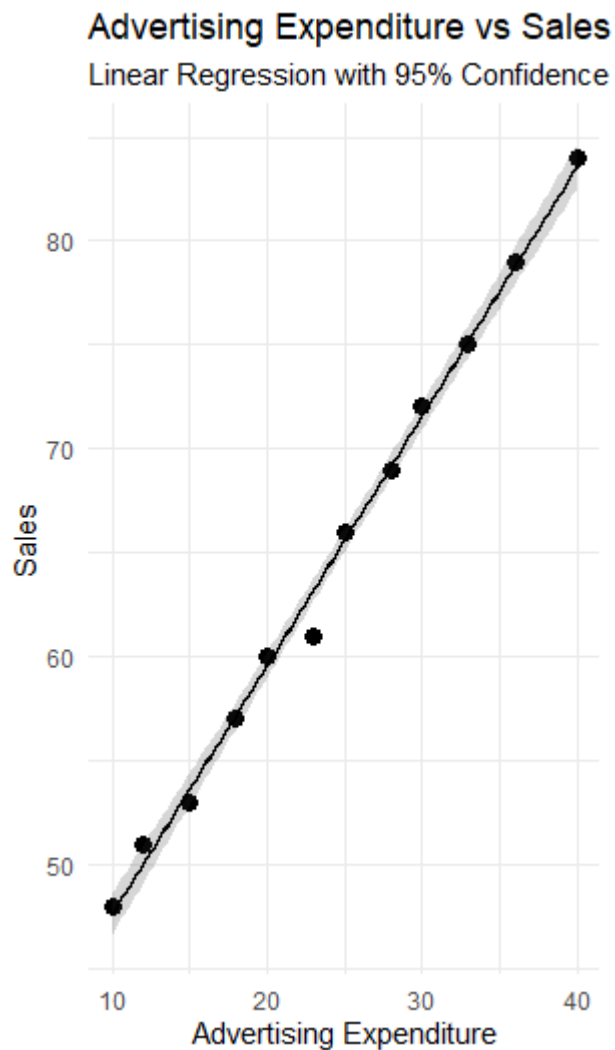
```
new_data$upr <- pred[, "upr"]
```

```
# Plot
```

```
ggplot(data_q2, aes(x = Advertising, y = Sales)) +
```

```
  geom_point(size = 3) +
```

```
geom_ribbon(  
  data = new_data,  
  aes(  
    x = Advertising,  
    ymin = lwr,  
    ymax = upr  
  ),  
  inherit.aes = FALSE,  
  alpha = 0.2  
) +  
geom_line(  
  data = new_data,  
  aes(  
    x = Advertising,  
    y = fit  
  ),  
  inherit.aes = FALSE,  
  linewidth = 1  
) +  
labs(  
  title = "Advertising Expenditure vs Sales",  
  subtitle = "Linear Regression with 95% Confidence Band",  
  x = "Advertising Expenditure",  
  y = "Sales"  
) +  
theme_minimal()
```



Regression equation

The fitted model is approximately:

$$Sales = 34.28 + 1.26(Advertising)$$

This means that for every one-unit increase in advertising expenditure, predicted sales increase by approximately **1.26 units**.

Interpretation

There is a strong positive relationship between advertising expenditure and sales.

As advertising expenditure increases, sales also increase. The fitted regression line shows the estimated trend.

The 95% confidence band represents uncertainty around the estimated **mean sales trend**.

How uncertainty changes

The confidence band is generally:

- narrower around the center of the observed advertising range,
- wider toward the lower and upper extremes.

This happens because there are fewer observations supporting the regression estimate at the extremes.

Important distinction

A **confidence interval** describes uncertainty about the estimated mean response.

A **prediction interval** would be wider because it also includes individual-observation variability.

Visualization pitfall

The confidence band should not be interpreted as saying that every future observation will fall inside the band. It represents uncertainty about the **mean regression line**, not individual future sales.

Q3. Overplotting, Jittering and Transparency

Given data

```
Hours <- c(
  2,3,3,4,4,4,5,5,5,5,
  6,6,6,6,6,7,7,7,7,7,
  8,8,8,8,8,9,9,9,10,10
)

Scores <- c(
  45,52,52,58,60,60,64,66,66,66,
  70,72,72,72,72,72,76,78,78,80,80,
  84,84,85,85,85,88,90,90,94,94
)
```

Plot 1 — Original scatterplot

Q3: Overplotting

```
library(ggplot2)
```

```
Hours <- c(
  2,3,3,4,4,4,5,5,5,5,
  6,6,6,6,6,7,7,7,7,7,
  8,8,8,8,8,9,9,9,10,10
)
```

```
Scores <- c(
  45,52,52,58,60,60,64,66,66,66,
  70,72,72,72,72,76,78,78,80,80,
  84,84,85,85,85,88,90,90,94,94
)
```

```
data_q3 <- data.frame(Hours, Scores)
```

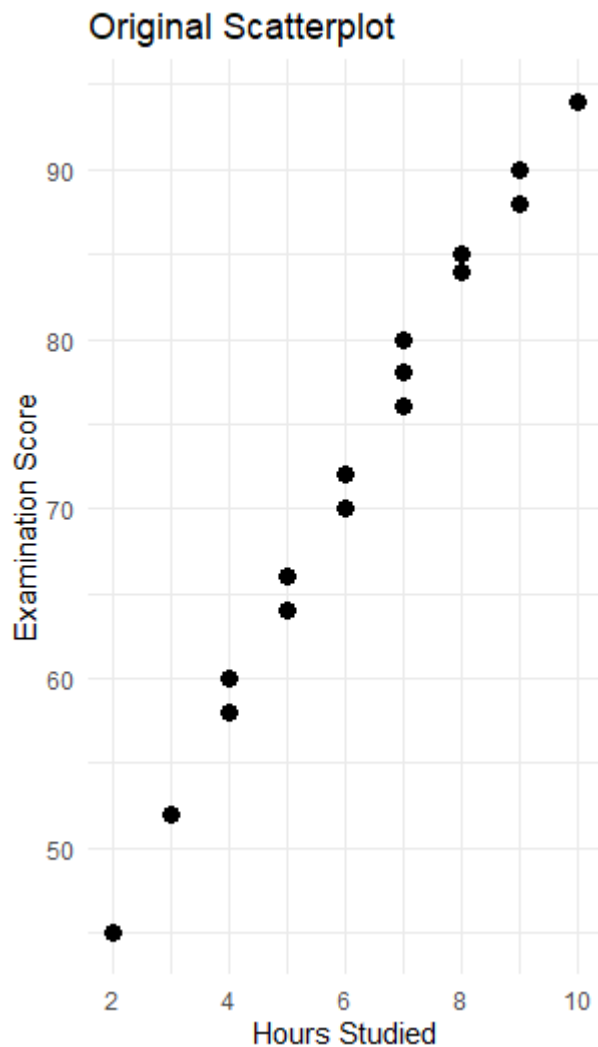
```
# Original scatterplot
```

```
ggplot(data_q3, aes(x = Hours, y = Scores)) +
  geom_point(size = 3) +
  labs(
    title = "Original Scatterplot",
    x = "Hours Studied",
    y = "Examination Score"
  ) +
  theme_minimal()
```

Plot 2 — Jittering and transparency

```
# Redesigned scatterplot
```

```
ggplot(data_q3, aes(x = Hours, y = Scores)) +  
  geom_jitter(  
    width = 0.15,  
    height = 0.5,  
    alpha = 0.5,  
    size = 3  
  ) +  
  labs(  
    title = "Scatterplot with Jittering and Transparency",  
    x = "Hours Studied",  
    y = "Examination Score"  
  ) +  
  theme_minimal()
```



Comparison

Original plot

Several students have identical values. When points overlap, multiple observations may appear as one point.

This is called **overplotting**.

For example, if five students have the same:

Hours = 6

Score = 72

the plot may visually show only one point.

Redesigned plot

Jittering slightly moves the points so that overlapping observations can be distinguished.

Transparency also helps because darker regions indicate multiple overlapping observations.

Interpretation

There is a clear positive association between hours studied and examination scores.

Students who study more hours generally achieve higher examination scores.

Visualization pitfall

Without jittering or transparency, the number of observations can be underestimated because overlapping points are hidden.

Important point

Jitter should be small enough that it does not create a false relationship. It is only used to reveal overlapping observations.

Q4. 2D Histogram and Density Estimation

Given data

Height = 150,152,155,158,160,161,163,165,166,168,
170,171,173,175,176,178,180,182,184,186

Weight = 48,50,53,55,57,59,60,62,64,65,
68,69,71,73,74,77,80,82,85,88

R Code

```
# Q4: 2D Histogram and Density
```

```
library(ggplot2)
```

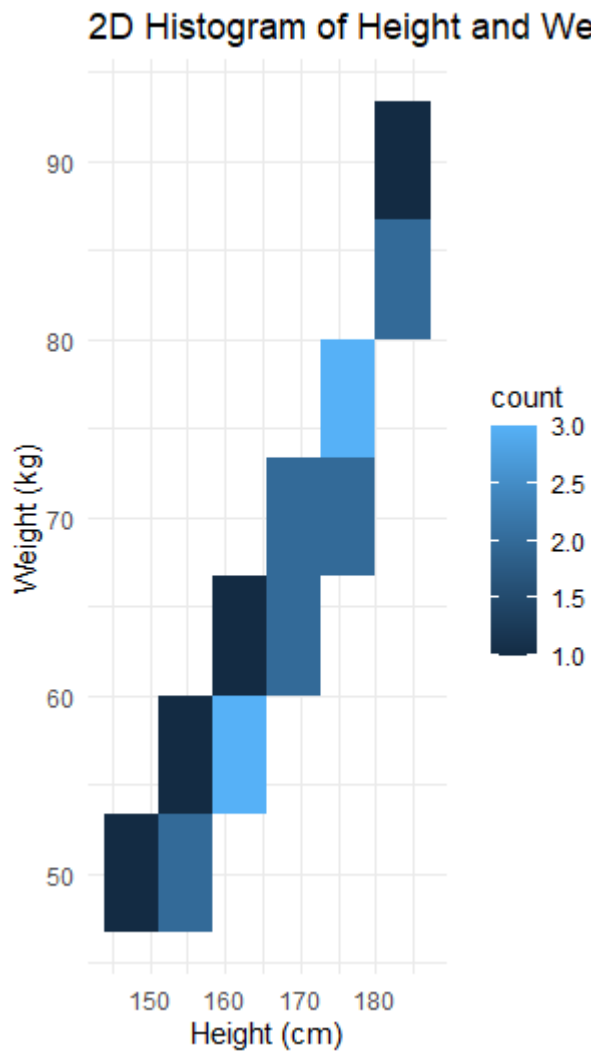
```
Height <- c(  
  150,152,155,158,160,161,163,165,166,168,  
  170,171,173,175,176,178,180,182,184,186  
)
```

```
Weight <- c(
  48,50,53,55,57,59,60,62,64,65,
  68,69,71,73,74,77,80,82,85,88
)
```

```
data_q4 <- data.frame(Height, Weight)
```

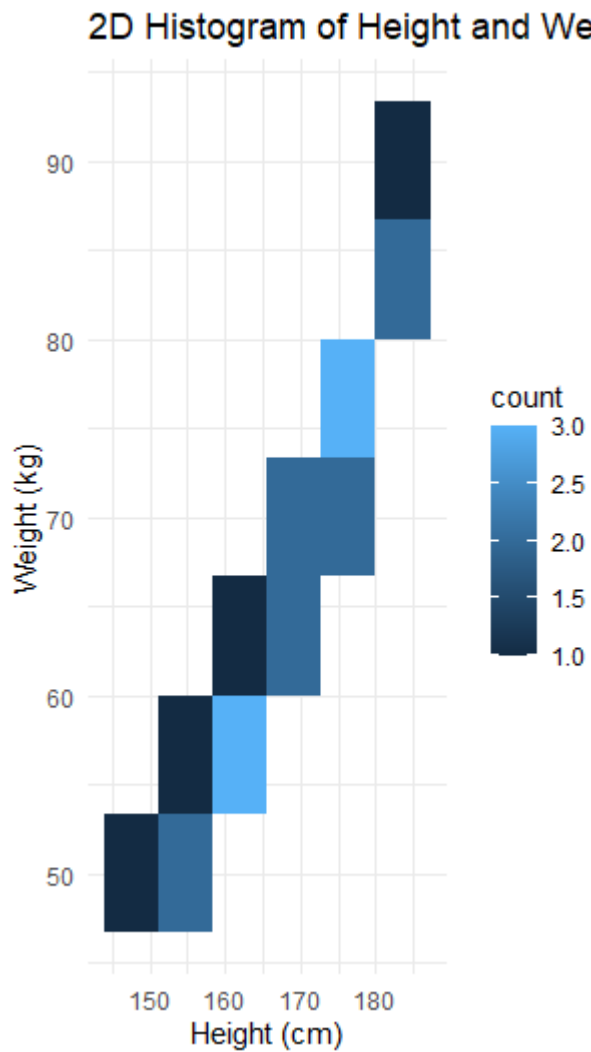
2D Histogram

```
ggplot(data_q4, aes(x = Height, y = Weight)) +
  geom_bin2d(bins = 6) +
  labs(
    title = "2D Histogram of Height and Weight",
    x = "Height (cm)",
    y = "Weight (kg)"
  ) +
  theme_minimal()
```



2D Density Visualization

```
ggplot(data_q4, aes(x = Height, y = Weight)) +  
  geom_point(alpha = 0.4) +  
  geom_density_2d() +  
  labs(  
    title = "2D Density Estimation of Height and Weight",  
    x = "Height (cm)",  
    y = "Weight (kg)"  
  ) +  
  theme_minimal()
```



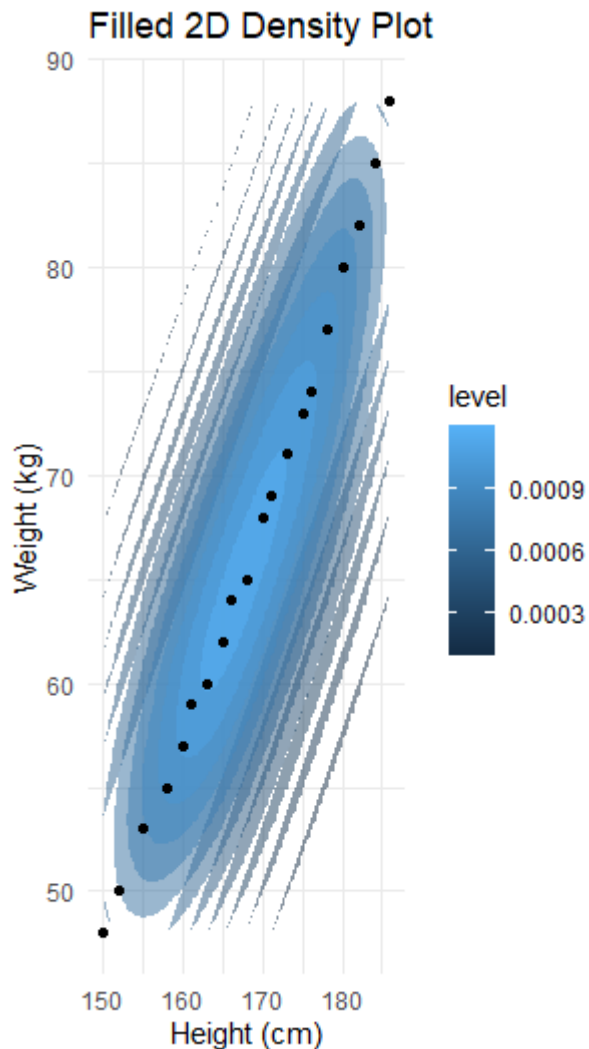
Alternative filled density plot

```
ggplot(data_q4, aes(x = Height, y = Weight)) +
  stat_density_2d(
    aes(fill = after_stat(level)),
    geom = "polygon",
    alpha = 0.5
  ) +
  geom_point() +
  labs(
    title = "Filled 2D Density Plot",
    x = "Height (cm)",
```

```
y = "Weight (kg)"
```

```
) +
```

```
theme_minimal()
```



Interpretation

The observations show a strong positive relationship between height and weight.

As height increases, weight generally increases.

The highest concentration of observations is around the **middle portion of the height-weight range**, approximately:

- Height: **160–175 cm**
- Weight: **57–74 kg**

2D Histogram

The 2D histogram divides the plotting area into rectangular bins. Darker/more concentrated bins represent regions containing more observations.

2D Density

The density plot estimates the continuous concentration of observations.

The contours indicate areas having similar estimated density.

Which communicates concentration more clearly?

The **2D density visualization** communicates concentration more clearly when the objective is to understand the overall shape and concentration of the joint distribution.

The 2D histogram is easier to understand for counting observations within rectangular regions, while the density plot provides a smoother representation of concentration.

Visualization pitfall

Density estimation depends on the bandwidth. An inappropriate bandwidth can make the distribution appear either:

- too smooth, hiding important patterns, or
- too irregular, exaggerating random variation.

Q5. Color Scales, Accessibility and Misleading Visualization

Given data

Category = A,B,C,D,E,F,G

Satisfaction = 42,48,55,61,67,74,82

The data are ordered from low satisfaction to high satisfaction.

Therefore, an ordered **sequential color scale** is appropriate.

Plot 1 — Potentially misleading diverging/nonmonotonic color scale

Q5: Misleading Color Scale

```
library(ggplot2)
```

```
Category <- c("A", "B", "C", "D", "E", "F", "G")
```

```
Satisfaction <- c(  
  42, 48, 55, 61, 67, 74, 82  
)
```

```
data_q5 <- data.frame(  
  Category,  
  Satisfaction  
)
```

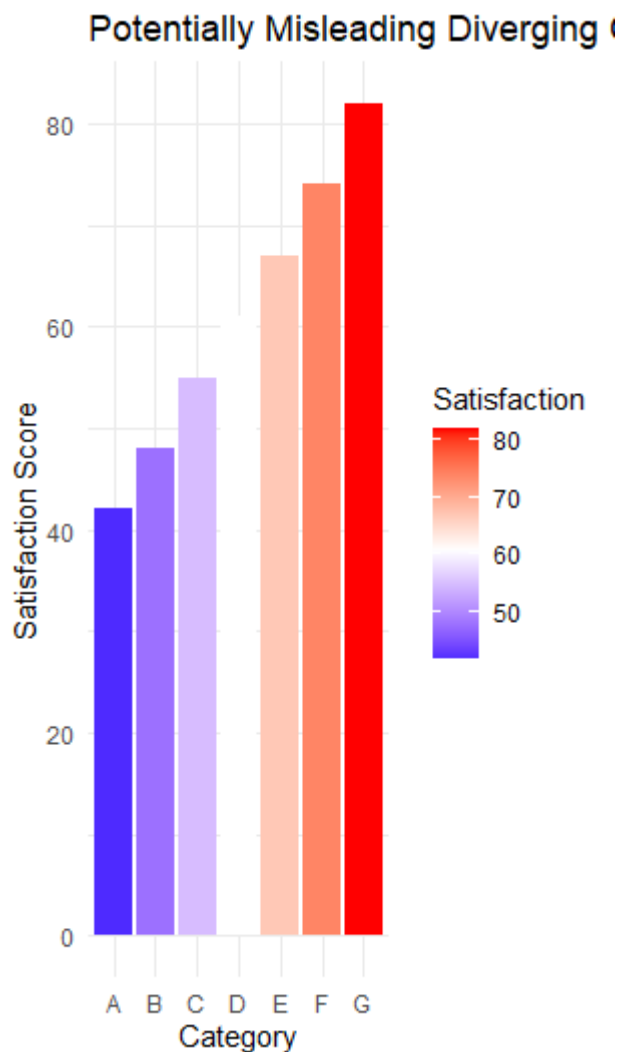
```
# Misleading/nonmonotonic color scale
```

```
ggplot(  
  data_q5,  
  aes(  
    x = Category,  
    y = Satisfaction,  
    fill = Satisfaction  
  )  
) +  
  geom_col() +  
  scale_fill_gradient2(  
    low = "blue",  
    mid = "white",  
    high = "red",  
    midpoint = 61  
  ) +  
  labs(  
    title = "Satisfaction by Category",  
    x = "Category",  
    y = "Satisfaction",  
    fill = "Satisfaction"  
  )
```

```

title = "Potentially Misleading Diverging Color Scale",
x = "Category",
y = "Satisfaction Score",
fill = "Satisfaction"
) +
theme_minimal()

```



Why can this be misleading?

A diverging scale is designed for data with a meaningful central reference point.

For example:

Below target → Target → Above target

But the satisfaction scores here simply form an ordered range from **42 to 82**.

There is no meaningful reason for the value 61 to act as a special midpoint.

Using blue → white → red may therefore make values around 61 appear qualitatively different from values slightly above or below it.

The color change can suggest a meaningful boundary that does not actually exist.

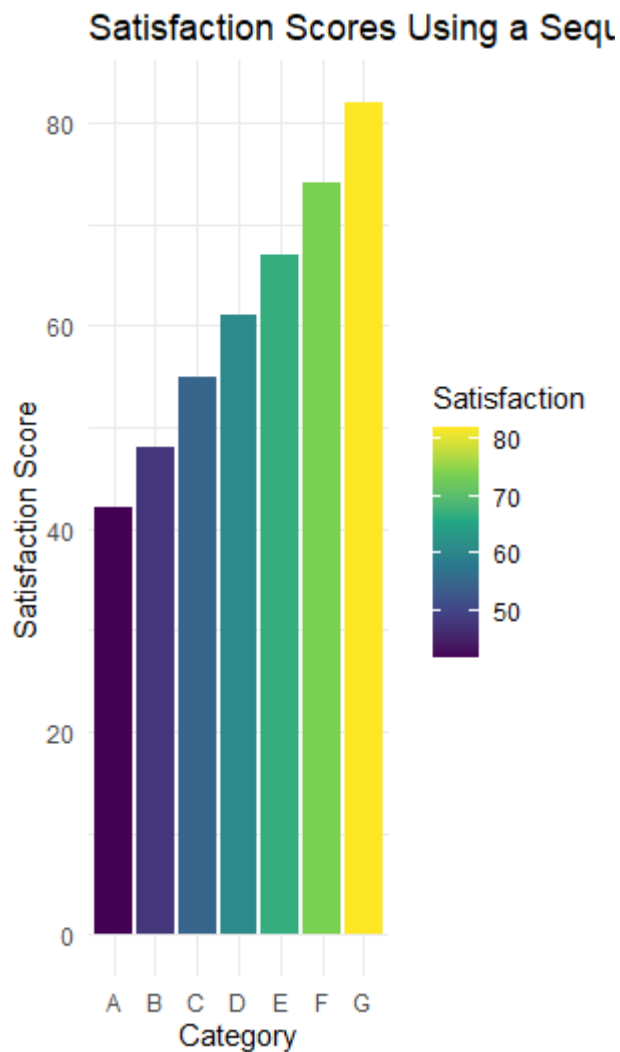
Plot 2 — Sequential and accessibility-friendly redesign

A sequential scale is more appropriate because satisfaction increases continuously.

Accessibility-friendly sequential scale

```
ggplot(  
  data_q5,  
  aes(  
    x = Category,  
    y = Satisfaction,  
    fill = Satisfaction  
  )  
) +  
  geom_col() +  
  scale_fill_viridis_c(  
    option = "viridis",  
    direction = 1  
  ) +  
  labs(  
    title = "Satisfaction Scores Using a Sequential Color Scale",  
    x = "Category",  
    y = "Satisfaction Score",  
    fill = "Satisfaction"  
  ) +
```

theme_minimal()



Interpretation

The satisfaction score increases steadily from:

Category A = 42

to:

Category G = 82

The redesigned plot uses progressively changing colors to represent increasing satisfaction.

The color intensity/order corresponds consistently to the numerical values.

Why is the second design better?

The second visualization is better because:

1. **It preserves numerical ordering.**

2. It does not introduce an artificial midpoint.
3. The sequential scale naturally represents low → high values.
4. The viridis scale is designed to remain relatively interpretable for people with common color-vision deficiencies.
5. The color encoding supports rather than distracts from the quantitative comparison.

Visualization pitfall in the first plot

The main pitfall is using a **diverging/nonmonotonic color scheme for data without a meaningful midpoint**.

This can exaggerate differences and imply categories or thresholds that do not exist.